

# SYSTEM SOFTWARE

## (UNIT-2)

### ASSEMBLER

An **assembler** is a system software program that **translates assembly language source code into machine language (object code)** that the computer can execute.

The assembler converts these human-readable instructions into their corresponding **machine-code/object-code representation**.

#### What the Assembler Does

1. **Reads the source program** written in assembly language.
2. **Identifies instructions and symbols** such as LDA, ADD, ALPHA, etc.
3. **Converts mnemonics into machine opcodes**.
4. **Calculates addresses** of labels and variables.
5. **Generates object code**.
6. Produces information such as a **symbol table** and may report **errors**.

### DESIGN PROCEDURES OF AN ASSEMBLER

The design of an assembler is carried out systematically so that the assembler can correctly translate an **assembly language program into machine/object code**.

The main steps are:

#### 1. Specify the Problem Properties

First, clearly identify **what the assembler needs to do** and the properties of the target machine and assembly language.

Important properties include:

- Instruction set and opcodes
- Instruction formats
- Addressing modes

- Registers
- Memory organization
- Data formats
- Assembly language syntax
- Assembler directives such as START, END, DS, DC, and LTORG
- Type of object program to be generated

**In simple words:**

**Find out what the assembler must understand and what output it must produce.**

## **2. Specify the Data Structures**

Next, decide what **tables and data structures** are required to store information while assembling the program.

Important data structures include:

### **SYMTAB — Symbol Table**

Stores symbols and their addresses.

### **LITTAB — Literal Table**

Stores literals and their addresses.

### **OPTAB — Opcode Table**

Stores operation codes and their machine codes.

## **3. Specify the Format of the Data Structures**

After deciding which data structures are needed, specify **exactly how information will be stored inside them.**

The format should make it easy and fast for the assembler to **insert, search, and retrieve information.**

**In simple words:**

**Decide what fields each table will contain and how the information will be organized.**

## **4. Design the Algorithm**

Now design the **step-by-step algorithm** that the assembler will follow.

The algorithm generally includes:

### Pass 1

- Read the source program.
- Initialize the Location Counter.
- Identify labels/symbols.
- Enter symbols into SYMTAB.
- Identify literals and enter them into LITAB.
- Process assembler directives.
- Generate intermediate code.

### Pass 2

- Read intermediate code.
- Search OPTAB for machine opcodes.
- Search SYMTAB/LITAB for addresses.
- Generate the final object code.
- Report errors.

**In simple words:**

**Decide exactly how the assembler will perform its work.**

### 5. Look for Modularity

The assembler should be divided into **small, independent modules**, rather than making one huge program.

Advantages of modularity:

- Easier to understand
- Easier to test
- Easier to debug
- Easier to modify
- Modules can be reused

**In simple words:**

**Break the assembler into smaller, manageable parts.**

### 6. Repeat Steps 1 to 5

Assembler design is usually **iterative**.

After designing the initial version, review it and check:

- Are all machine features handled?
- Are all required data structures present?
- Is the algorithm correct?
- Can the modules be improved?
- Are there unnecessary or missing components?

If something is wrong or incomplete, go back and repeat **Steps 1–5** until the design is satisfactory.

## SYMBOL-DEFINING STATEMENTS & LITERALS — ASSEMBLER

In assembly language, **symbol-defining statements and literals** help the assembler assign names, reserve memory, define constants, and manage addresses.

These are mostly **pseudo-operations (pseudo-ops)**. Unlike machine instructions, pseudo-ops **do not directly execute on the CPU**. They give instructions to the assembler.

### 1. START

#### Meaning

START indicates the **starting address of the program**.

The assembler initializes the **Location Counter (LOCCTR)** to this address.

#### Important point

START **does not generate object code**.

### 2. END

#### Meaning

END indicates the **end of the source program**.

It tells the assembler that there are no more source statements to process.

The operand can specify the **address of the first executable instruction**.

#### Important point

END does not generate normal machine instructions.

### 3. USING

#### Meaning

It tells the assembler which **base register** is being used to calculate addresses.

This tells the assembler to use **register 15 as a base register** for address calculation.

#### Why is USING needed?

If an instruction uses a symbol, the assembler needs to know how to calculate its address using the available base registers.

#### Important point

USING is a **pseudo-op**. It does not perform a CPU operation itself.

### 4. DROP

#### Meaning

DROP tells the assembler to **stop using a particular register as a base register**.

#### USING vs DROP

| USING                                         | DROP                                                |
|-----------------------------------------------|-----------------------------------------------------|
| Makes a register available as a base register | Removes a register from base-register consideration |
| Establishes address association               | Cancels that association                            |
| Used for address calculation                  | Stops that association                              |

### 5. DS — Define Storage

#### Meaning

DS stands for **Define Storage**.

It is used to **reserve memory space** for variables or data.

#### Common DS forms

Depending on the assembler, different storage types may be available:

- DS F → Fullword storage
- DS H → Halfword storage
- DS D → Doubleword storage
- DS C → Character storage

- DS X → Hexadecimal storage

### Important point

**DS allocates/reserves memory but does not normally place a specified constant value into that memory.**

## 6. DC — Define Constant

### Meaning

DC stands for **Define Constant**.

It is used to define a constant and **allocate storage containing that value**.

### DS vs DC

This is very important for exams:

| DS                                         | DC                                                        |
|--------------------------------------------|-----------------------------------------------------------|
| Define Storage                             | Define Constant                                           |
| Reserves memory                            | Reserves memory and initializes it with a specified value |
| Does not specify an initial constant value | Specifies the value to be stored                          |
| Example: NUM DS F                          | Example: NUM DC F'10'                                     |

### Easy trick

**DS = Space**

**DC = Value**

## 7. LTORG — Literal Pool

### Meaning

LTORG stands for **Literal Origin**.

It tells the assembler to **place the literals used in the program into memory at that point**.

### What is a literal?

A **literal** is a constant written directly as part of an instruction.

### Literal Pool

The collection of literals stored together in memory is called a **literal pool**.

The assembler assigns addresses to these literals.

### Why use LTORG?

It helps control **where literals are placed in memory**.

If no LTORG is used, the assembler may place pending literals at the end of the program (depending on assembler rules).

### Important points

- LTORG does not execute on the CPU.
- It causes the assembler to create a **literal pool**.
- It assigns addresses to pending literals.
- It helps keep literals within an appropriate addressing range.

## 8. EQU — Equal Operator

### Meaning

EQU stands for **EQUate**.

It assigns a **symbolic name to a value or address**.

### Important point

EQU **does not reserve memory**.

It simply defines a symbolic value.

## 9. Symbol-Defining Statements

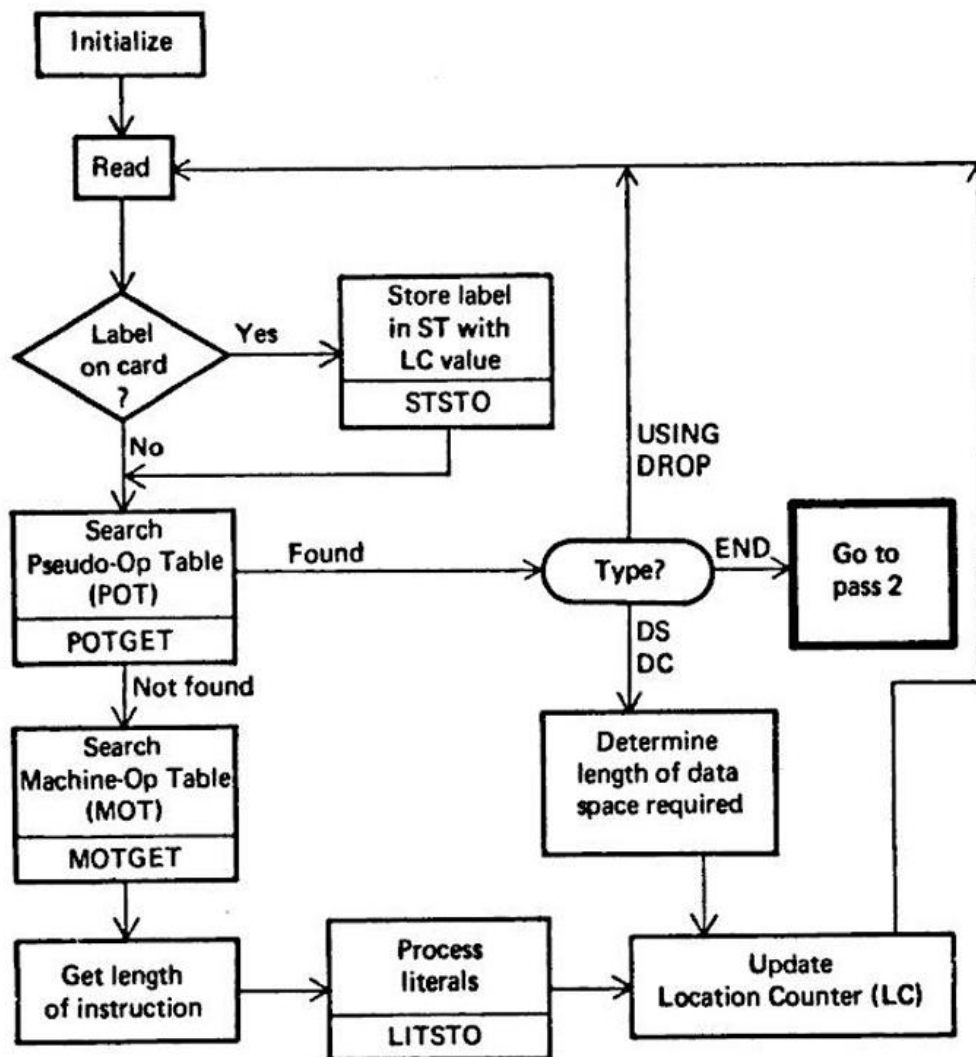
A **symbol** is a name used to represent an address or value.

## TWO-PASS ASSEMBLER DESIGN

A **two-pass assembler** is an assembler that processes the source program **twice** to convert assembly language into machine/object code.

The main reason for using two passes is to handle **forward references**—when a symbol is used before its address has been defined.

### Pass 1 of a Two-Pass Assembler



**FIGURE 3.3** Pass 1 overview: define symbols

**What is Pass 1?**

**Pass 1 = Find addresses and define symbols.**

Its main jobs are:

1. Read the assembly program.
2. Maintain the **Location Counter (LC)**.
3. Find labels/symbols and store their addresses in **Symbol Table (ST)**.
4. Process literals and store them in **Literal Table (LT)**.
5. Calculate how much memory each instruction/data needs.
6. Handle assembler directives like **START, USING, DROP, DS, DC, END**.

👉 **Pass 1 does NOT generate the final machine/object code.**

## Understanding the flowchart

### Step 1: Initialize

The assembler starts by initializing its tables and the **Location Counter (LC)**.

### Step 2: Read

The assembler reads **one statement/instruction at a time** from the source program.

### Step 3: Is there a label?

The assembler checks:

#### Does this statement have a label?

If **YES**, store it in the **Symbol Table (ST)** along with the current LC value.

### Step 4: Search the Pseudo-Op Table (POT)

If there is no label, or after processing the label, the assembler checks:

Is this a **Pseudo Operation**?

## ◆ What happens if it is a Pseudo-Op?

The flowchart asks:

### Type?

There are different types.

#### 1. END

If the statement is:

END

Pass 1 is finished.

➡ **Go to Pass 2**

#### 2. USING / DROP

These directives are related to **base register management**.

They don't normally allocate new program storage like DS or DC.

So the assembler processes them and continues reading.

### 3. DS / DC

These are important because they **reserve or allocate memory**.

#### **DS — Define Storage**

It tells the assembler:

Reserve space for NUM.

#### **DC — Define Constant**

It tells the assembler:

For both DS and DC, the assembler must determine:

#### **How much memory is required?**

Then it updates the LC.

#### **Search Machine-Op Table (MOT)**

If the instruction is **not a pseudo-op**, the assembler searches the **Machine Operation Table (MOT)**.

Examples of machine instructions:

**L** → Load (transfer data from memory into a register)

**A** → Add (add contents of memory to a register)

**ST** → Store (save contents of a register into memory)

**B** → Branch (unconditional jump to another instruction address)

**BALR** → Branch and Link Register (save return address and branch, used for subroutine calls)

**BR** → Branch Register (branch to the address contained in a register)

**BCT** → Branch on Count (decrement a register and branch if not zero)

The MOT tells the assembler information such as:

- instruction mnemonic
- instruction type
- instruction length

#### **Get length of instruction**

Once the assembler finds the instruction in MOT, it determines its length.

## ◆ Process Literals

The assembler also checks for **literals**.

The assembler stores it in the **Literal Table (LITTAB)**.

## ◆ Update Location Counter (LC)

After processing the instruction/data, the assembler updates the LC.

**Simple formula:**

**New LC = Old LC + Required memory space**

*Most important tables*

For your exam, remember these three:

### 1. MOT — Machine Operation Table

Contains **machine instructions**.

Example:

L, A, ST, B, BR

It helps determine **instruction length and machine operation information**.

### 2. POT — Pseudo-Operation Table

Contains **assembler directives**.

Example:

**START** → Marks the beginning of a program and specifies the starting address.

**END** → Indicates the end of the source program to the assembler.

**USING** → Establishes a base register for addressing, telling the assembler which register holds a base address.

**DROP** → Cancels the effect of a USING directive, i.e., releases a base register from use.

**DS** → Define Storage — reserves memory space without initializing it.

**DC** → Define Constant — allocates memory and initializes it with specified constant values.

These instructions/directives are handled by the **assembler**, not directly executed by the CPU.

### 3. ST — Symbol Table

Stores:

## Symbol/Label + Address

Example:

### Symbol Address

LOOP 1000

NUM 1010

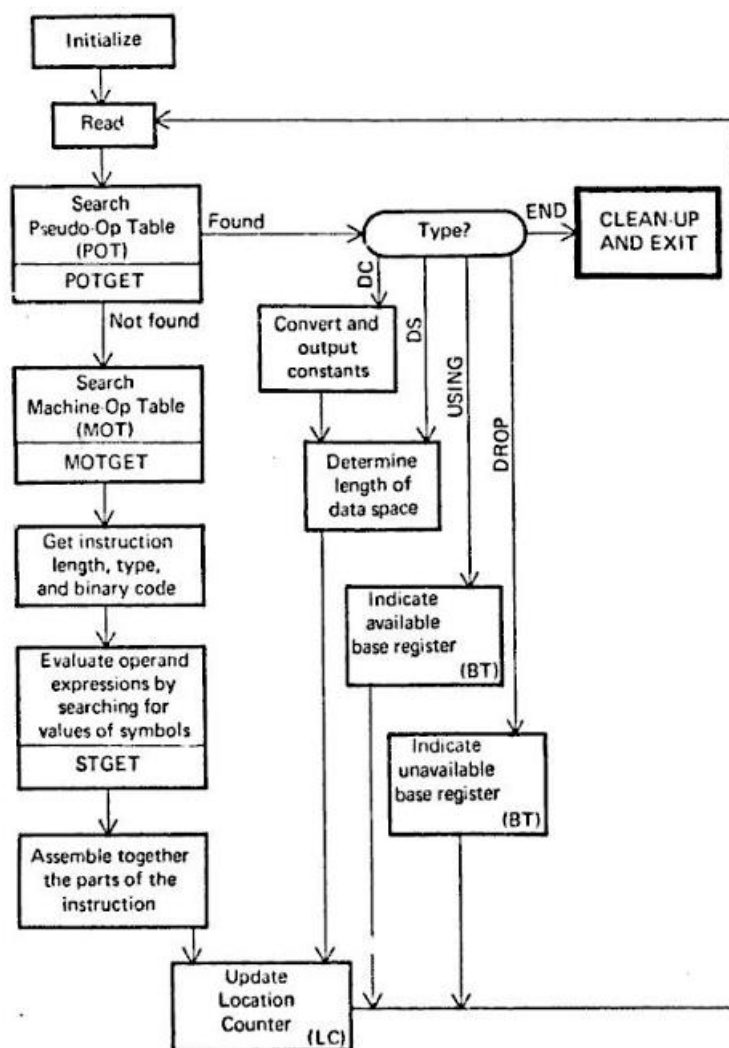
TOTAL 1014

## 4. LITTAB — Literal Table

Stores:

### Literal + Assigned Address

## PASS 2 FLOWCHART OF A TWO-PASS ASSEMBLER



Pass 2 overview: evaluate fields and generate code

So remember:

- **Pass 1 → Find addresses and create tables**
- **Pass 2 → Use those addresses to generate machine/object code**

Let's understand the flowchart in very easy language.

### 🕒 What does Pass 2 do?

Pass 2 takes the source program again and uses the information created during Pass 1:

- **Symbol Table (ST)**
- **Literal Table (LT)**
- **Machine Operation Table (MOT)**
- **Pseudo-Operation Table (POT)**

Its main job is to generate the **final machine code/object code**.

### ◆ Step-by-step flowchart

#### 1. Initialize

First, Pass 2 initializes the required tables and the **Location Counter (LC)**.

Initialize



Read

#### 2. Read a statement

The assembler reads one statement/instruction from the source program.

#### 3. Search Pseudo-Operation Table (POT)

First, it checks:

**Is this a pseudo-operation?**

It searches the **POT**.

**If it is a Pseudo-Op → Check its type**

The flowchart asks:

**Type?**

There are four important possibilities shown.

## □ A. DC — Define Constant

DC means **Define Constant**.

The assembler:

1. Converts the constant into its machine/binary representation.
2. Outputs the converted constant.
3. Determines how much memory it occupies.
4. Updates the LC.

**Easy meaning:**

**DC → Convert the value and put it into object code.**

## □ B. DS — Define Storage

DS means **Define Storage**.

It tells the assembler:

Reserve some memory space.

The assembler determines the required space.

**Important:**

**DS usually does not generate actual data.**

It only reserves space.

## □ C. USING

USING tells the assembler which **base register** can be used for address calculation..

The flowchart represents this as:

**Indicate available base register (BT)**

where **BT = Base Table**.

## ● D. DROP

DROP tells the assembler:

Stop using a particular register as a base register.

Therefore, the assembler marks that register as:

**Unavailable base register**

This is shown as:

**Indicate unavailable base register (BT)**

☆ **What if it is NOT a Pseudo-Op?**

Then the assembler searches the:

### **Machine Operation Table (MOT)**

The MOT contains information about the instruction.

#### ◆ **5. Get instruction length, type and binary code**

After finding the instruction in MOT, the assembler gets:

- **Instruction length**
- **Instruction type**
- **Binary/machine opcode**

Opcode = machine code for L

Type = particular instruction format

#### ◆ **6. Evaluate operand expressions**

This is a very important step.

The assembler examines the operands and looks up the values of symbols using the **Symbol Table (ST)**.

The flowchart shows:

#### **Evaluate operand expressions by searching for values of symbols — STGET**

Pass 2 replaces the symbolic reference NUMBER with its actual address information.

So it can generate the correct machine instruction.

#### ◆ **7. Assemble the parts of the instruction**

Now the assembler combines all the required parts:

- Opcode
- Register information
- Address/displacement
- Other instruction fields

conceptually:

Opcode + Register + Address



Machine instruction

This is the actual **code generation** stage.

### ◆ 8. Update Location Counter

After generating/processing the instruction, the assembler updates the LC.

#### **What happens when END is found?**

If the assembler encounters:

END

the flow goes to:

#### **CLEAN-UP AND EXIT**

That means Pass 2 is finished.

The assembler performs necessary final operations and exits.